Programs for a Frozen Machine: The Description Length of Language and Task in Optimized Prefixes

Georgii Sirotenko
Heidelberg University
georgii.sirotenko@stud.uni-heidelberg.de

Abstract

A frozen language model is a fixed machine; a soft prefix that makes it perform a task is a program for that machine, and the shortest such program is a model-relative description length of the task. We measure that length for a frozen Qwen2.5-1.5B base model over a grid of three tasks (translation from English, factual recall, sentence continuation) and three output languages (Russian, German, English), in two units: prefix slots and variational bits under a bits-back code. Four findings. (i) Slots are not a description length: translation saturates at one slot, and one trained slot beats the natural-language prompt everywhere. (ii) In bits, a translation program costs about 500 bits at 10% excess loss, a Russian recall program five times that, and English recall most of all. (iii) Neither task nor language is the cheap modifier of the other, and sharing does not compress: a language given a task costs what a task given a language costs, and a hierarchy with one centre per task and a deviation per language costs 1.5 times eight independent programs. The asymmetry is in the mechanism: translation programs are read in the first block, recall programs deep in the network. (iv) The variational objective selects compact programs but the standard optimiser does not find them: a conservative step size and checkpoint selection on the full objective lower the measured frontier by 20–71%. All numbers regenerate from the released tables.

1 Introduction

Prompt tuning [8] and prefix tuning [9] adapt a frozen language model by optimising a few continuous vectors prepended to the input. Read as engineering, these vectors are parameters. Read through the lens of algorithmic information theory they are programs: the frozen model is the interpreter, the prefix is the program, the input is the data and the target is the result. Shaw et al. [12] make this reading precise, showing that prompt tokens form a program tape for a Transformer interpreter and that the variational code length of a prompt is an asymptotically optimal description-length objective. Cosma [2] define prompting complexity as a model-relative, resource-bounded Kolmogorov complexity.

This project measures the description length of the programs for multilingual tasks. For a frozen model M, task ρ and output language l, let $K_M(\rho, l)$ be the length of the shortest prefix that performs ρ in l at a chosen distortion. We ask:

- How is $K_M(\rho, l)$ split between telling the model which task to perform and which language to produce?
- Are the two parts separable, so that a task part and a language part can be assembled?
- How far apart are the programs for different cells, and do they cluster by task or by language?
- Where in depth does the model read each part?

Preprint. Under review.

- Is the measurement itself trustworthy: does gradient descent find the short programs that the objective prefers?

Our contributions are a measurement design and five results. The design (Section 3) fixes two units, slots and bits, one normalised distortion, two preregistered operating points, and a hierarchical variational prefix whose centre and deviations put task bits and language bits in the same currency. The results (Sections 5–6) are the four findings of the abstract plus a negative one: the information-distance matrix over cells has no cluster structure at one seed. Every number is recomputed from released tables by a notebook that runs in under a minute.

2 Related work

Description length of models and probes Hinton and van Camp [5] introduced the variational (bits-back) code for network weights; Blier and Ollivier [1] compared variational and prequential codes for deep networks and found the prequential one far shorter; Voita and Titov [14] used prequential coding to measure what a probe learns. We use both codes: variational for the rate of a prefix and prequential for the distance between prefixes.

Soft prompts as programs Shaw et al. [12] give the universality construction and the adaptive prior we build on, and report that the objective selects compressible solutions that random-initialisation training fails to find. Cosma [2] measure shortest prompts, discrete and soft, as a complexity. Mu et al. [10] compress instructions into gist tokens; Vu et al. [15] treat learned prompts as task embeddings that predict transfer; Vykopal et al. [16] combine soft language prompts with task adapters. Khashabi et al. [7] show that a working soft prompt need not read as words, which is why every claim here is functional: transfer, composition, conditioning or intervention, never a nearest-token reading.

Task and language inside the model Task vectors [4] and function vectors [13] show that a task can be carried by one hidden vector. Wendler et al. [17] and Dumas et al. [3] locate a language-agnostic concept representation in the middle blocks with language resolved later; our patching probes test where a learned task part and a learned language part are read.

3 Measurement framework

Interpreter, program, distortion The prefix $P \in \mathbb{R}^{k \times d}$ occupies the first k positions of the input sequence, where $d = 1536$ is the model's embedding dimension. The model's weights, tokenizer and format are fixed. Distortion is measured by the teacher-forced code length L of the target in bits per target token. Let m be the number of examples in the dataset being evaluated, (x_n, y_n) the input and target of example n, and N the total number of target tokens. For a prefix P, define the average negative log-likelihood as

$$L(P) = -\frac{1}{N} \sum_{n=1}^{m} \log_2 p_M(y_n \mid x_n, P),$$

where the probability of a target is the product over its tokens, each conditioned on the ground-truth preceding target tokens. In the training objective below, L and N refer to the training set; reported distortion uses held-out losses.

To compare across task–language pairs, we normalise the loss per pair:

$$\hat{D} = \frac{L - L_{\text{floor}}}{L_{\text{none}} - L_{\text{floor}}}, \tag{1}$$

where L_{none} is the no-prefix loss and $L_{\text{floor}} = \min\{L_{\text{LoRA}}, L_{\text{best prefix}}\}$ is the lower of two empirical reference losses: a per-pair rank-16 LoRA fine-tune of the attention projection matrices, with the pretrained weights frozen, and the best unpenalised prefix over the k sweep. All three losses are evaluated on the same dataset. The distortion thresholds $\hat{D} \leq 0.25$ and $\hat{D} \leq 0.10$ were chosen in advance from a development pilot and $\hat{D} \leq 0.05$ is reported as exploratory.

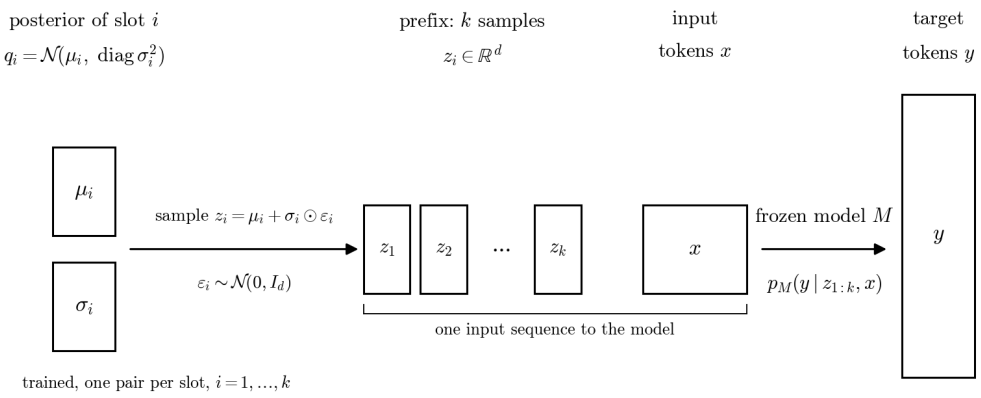


Figure 1: The sampling step of a variational prefix. Each slot i has a diagonal Gaussian posterior $q_i = \mathcal{N}(\mu_i, \mathrm{diag}\,\sigma_i^2)$ with trained μ_i and σ_i, and d is the model's embedding dimension (1536). At every forward call one sample $z_i = \mu_i + \sigma_i \odot \varepsilon_i$, with $\varepsilon_i \sim \mathcal{N}(0, I)$, is drawn per slot. The k samples form the prefix, which is prepended to the input tokens x and passed through the frozen model M.

Two units and rate–distortion objective *Slots* are the number k of prefix positions and the slot complexity k^* is the smallest k that reaches a distortion threshold, found by sweeping k with free, unpenalised vectors. *Bits* are the description length of the prefix under a prior: each slot i has a diagonal Gaussian posterior $q_i = \mathcal{N}(\mu_i, \mathrm{diag}\,\sigma_i^2)$, and the full prefix distribution is $q = \bigotimes_{i=1}^{k} q_i$, trained using the reparameterisation trick (as in VAEs; see Figure 1). We measure the expected description length of the sampled prefix, in bits, as $R(q) = D_{\mathrm{KL}}(q \,\|\, \pi_0)/\ln 2$, where π_0 is the product of independent per-slot diagonal Gaussian priors fitted to the model's token-embedding matrix (under an idealised bits-back coding scheme). The rate–distortion frontier of a task–language pair is the smallest description length among runs that reach a distortion threshold, over k and the KL weight β. Distortion of a variational prefix is scored on posterior samples, because the sample is the coded object (Section 6 shows what goes wrong when the posterior mean is scored instead).

The variational objective is

$$\mathcal{J}_\beta(q) = \mathbb{E}_{P\sim q}[L(P)] + \frac{\beta}{N}\frac{D_{\mathrm{KL}}(q\|\pi_0)}{\ln 2}.$$

Both terms are measured in bits per training target token. The first represents the expected data code length per token, while the second represents the prefix description length averaged over the training tokens. Multiplying the objective by N yields

$$N\mathcal{J}_\beta(q) = N\mathbb{E}_{P\sim q}[L(P)] + \beta R(q),$$

where $R(q) = D_{\mathrm{KL}}(q\|\pi_0)/\ln 2$ is the prefix description length. For $\beta = 1$, this corresponds to the idealised total expected description length under the assumed bits-back coding scheme, ignoring coding overheads. The expectation over $P \sim q$ is estimated with one sample per training step; reported distortion averages eight passes over the test set with a fresh sample per batch.

Hierarchy We test whether prefixes for different task–language pairs can be compressed by sharing a common component. For each group (either the languages of one task or the tasks of one language), we learn a shared centre c and a separate deviation δ_g for each member g, giving $P_g = c + \delta_g$. Both have the dimensions of an ordinary prefix, but may have different description lengths. Grouping by task shares a centre across languages and grouping by language shares one across tasks.

We learn Gaussian distributions q_c and q_g over the centre and deviations. The centre uses the original prior π_0, while each deviation uses a zero-centred Gaussian prior π_δ with the same covariance as π_0 ($\tau = 1$), so that a displacement costs the same bits in a deviation as in a whole prefix. The group's total description length is

$$R_{\mathrm{group}} = \frac{1}{\ln 2}\left(D_{\mathrm{KL}}(q_c\|\pi_0) + \sum_g D_{\mathrm{KL}}(q_g\|\pi_\delta)\right).$$

The centre is encoded only once, so sharing saves bits when this cost is smaller than that of independently encoding every member. A group reaches a distortion threshold only when every member does in the same run.

Training alternates between pairs within each group. For a pair g, we construct its prefix $P_g = c + \delta_g$ and minimise

$$\mathcal{J}_g = \mathbb{E}[L_g(c + \delta_g)] + \beta \left(\frac{R_c}{N_{\text{group}}} + \frac{R_g}{N_g} \right),$$

where R_c and R_g are the description lengths of the shared centre and the pair-specific deviation, N_g is the number of training target tokens for pair g, and $N_{\text{group}} = \sum_g N_g$.

Derived measurements We use four additional experiments to study how the learned prefixes represent tasks and languages.

Composition. We learn one prefix component per task and one per language, shared across task–language pairs. The components are concatenated to form a complete prefix. We first train the components jointly on all pairs. We then test whether they generalise to unseen combinations by excluding selected pairs from training and assembling their prefixes from components learned on other pairs: for example, exclude Russian translation from training, learn the translation component from German translation and the Russian component from Russian recall, concatenate them, and test whether the model translates into Russian without any additional training.

Distance. We measure the prequential cost $K(a \mid b)$ of training for pair a, initialised from pair b's prefix, and compare it with the from-scratch cost $K(a)$. The prequential cost is the predictive loss accumulated while learning, over a fixed budget of 512 examples in doubling blocks. The symmetric, normalised distance is

$$d(a, b) = \frac{K(a \mid b) + K(b \mid a)}{K(a) + K(b)}.$$

We also apply each trained prefix unchanged to other pairs to measure zero-shot transfer.

Mechanism. We investigate which transformer blocks use information from the learned prefix. We run the model with a trained prefix and record its activations, then run it with a random prefix of the same k. At one selected block, we replace only the activations at the prefix positions, at the input to self-attention, with those recorded from the trained-prefix run; all other activations remain unchanged. We repeat this for each block separately and measure how much of the trained prefix's improvement in loss each replacement recovers. Replacing at all blocks at once recovers it exactly.

Input compression (exploratory, Appendix C). We replace each input with a small number of trainable vectors specific to that example, and compare the resulting loss with and without a learned task prefix.

4 Experimental setup

Model, data, grid Qwen2.5-1.5B base [11], frozen, bfloat16 on an A100. Translation and continuation use FLORES+ (dev split for training and selection, devtest for reporting); recall uses mLAMA [6] with an English cloze prefix and the object rendered in the target language, split by item. Translation into English is excluded, so the grid has eight cells: `translation-{ru,de}`, `recall-{ru,de,en}`, `continuation-{ru,de,en}`. Continuation is unmeasurable: its floor lies within 0.10–0.16 bits per token of the no-prefix loss, below the seed variation, so it is kept in the tables and excluded from every claim.

Runs Design A: $k \in \{1, 2, 4, 8, 16, 32\}$, two seeds, 1500 steps, batch 16, AdamW with cosine decay; repeated with the target language given, the task given, and the natural-language prompt with no prefix; per-cell LoRA floors. Flat variational: $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$. Hierarchical: both nestings, $k \in \{2, 4, 8\}$, $\beta \in \{0.3, 1, 3\}$. Composition: 1+1, 2+2, 4+4 slots with five hold-out sets. Distance: every cell's $k = 4$ prefix toward every cell. Mechanism: eight cells, five probes, 28 blocks. In total 1,889 result rows and about 36 A100 hours (Appendix D).

Two variational protocols The preregistered protocol (*flat*) used learning rate 0.05, the checkpoint with the lowest development distortion at the posterior mean, and σ initialised at five times the prior. Its training histories (Section 6) showed that this overshoots the rate and selects on the wrong

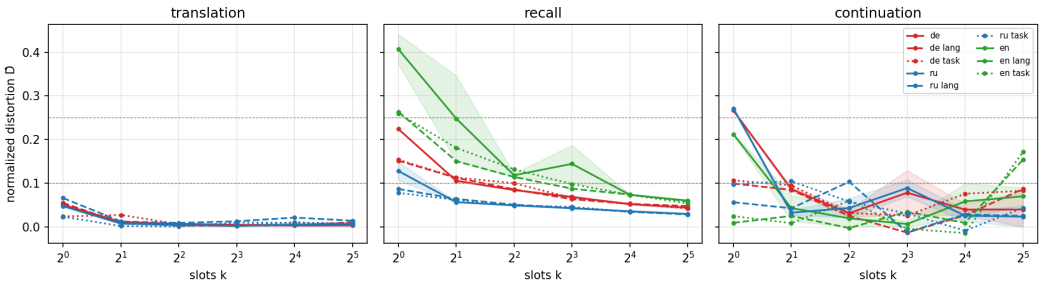


Figure 2: Normalised distortion against the number of slots, Design A, seed means with the seed range shaded. Dashed lines mark the operating points; line style is the conditioning format. Continuation (right) is noise.

cell	$k^*(0.25)$	$k^*(0.10)$	$k^*(0.05)$, expl.
translation-ru	1	1	1
translation-de	1	1	2
recall-ru	1	2	4
recall-de	1	4	32
recall-en	2	16	not reached

Table 1: Slot complexity: the smallest k whose seed-mean $\hat{D}$ reaches the level.

statistic. The corrected protocol (*flat2*) uses learning rate 0.01, scores development distortion on samples, selects the checkpoint with the lowest objective $\mathcal{J}_\beta$, and initialises σ at the prior. It was run on the five measurable cells at $k \in \{2, 4\}$, $\beta \in \{1, 3\}$ and is the headline where it exists. Both hierarchical nestings were rerun under it, and the corrected runs are the headline for every hierarchical comparison.

5 Results

5.1 Slots are not a description length

Figure 2 shows $\hat{D}$ against k. Translation reaches $\hat{D} \leq 0.10$ with one slot in every language and conditioning and is flat from there. Recall keeps improving to $k = 32$ on Russian and German with the budget binding: a 3000-step run on `recall-ru` at $k = 8$ moves $\hat{D}$ from 0.043 to 0.032 and is still falling. Seed gaps are below 0.011 on translation, 0.042 on `recall-ru` and `recall-de`, and 0.20 on `recall-en`. Table 1 gives k^*.

Two observations make the slot count unusable as a description length. First, the natural-language prompt, scored as a context with no prefix, is far worse than one trained slot: on recall it leaves $\hat{D}$ at 0.91–0.96 where one slot leaves 0.13–0.41; on translation 0.12–0.20 against 0.05. One vector of 1,536 numbers carries more than an instruction, so a token is not a unit of information. Second, conditioning saves little on the slot axis: giving the language or the task changes $k^*(0.10)$ in none of the translation cells and in two recall cells (`recall-ru` 2 to 1, `recall-en` 16 to 8, for either conditioning). The savings Δk_{lang} and Δk_{task} average 1.8 slots each over the measurable cells, with 8 of those 9 slots from `recall-en`. The slot axis cannot distinguish telling the model the language from telling it the task; the bit axis can.

5.2 Description length in bits

Table 2 is the headline. A translation program costs about 500 bits. A Russian recall program costs five times that; German and English recall do not reach 10% excess loss within the budget and cost 3,300 and 7,200 bits at 25%. A second seed reproduces the corrected column within 4% on three cells and within 22% on the other two, with the same ordering (Table 7).

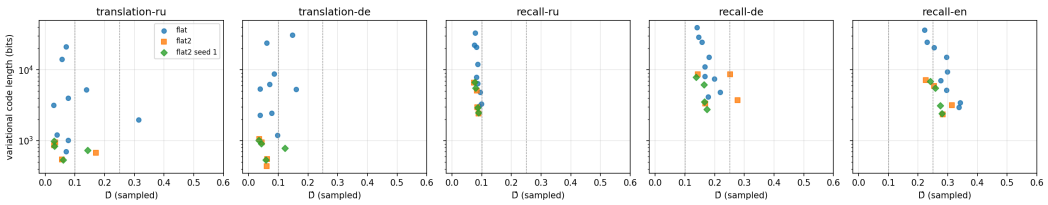


Figure 3: Rate against distortion for every variational run, per cell. Circles: preregistered protocol; squares: corrected protocol. Distortion is scored on posterior samples.

	corrected (flat2)		best	prereg.
cell	$\hat{D} \leq 0.10$	$\hat{D} \leq 0.25$	$\hat{D}$	$\hat{D} \leq 0.25$
translation-de	436	436	0.036	1,194
translation-ru	542	542	0.030	699
recall-ru	2,419	2,419	0.075	3,101
recall-de	—	3,343	0.143	4,168
recall-en	—	7,155	0.226	24,686

Table 2: Headline frontier: the smallest code length in bits among runs reaching each operating point, under the corrected protocol, with the best distortion any corrected run reaches and the preregistered protocol's value at 0.25.

The ordering across tasks within a language follows the learning gap $L_{\text{none}} - L_{\text{floor}}$ (recall 3.5–6.2 bits per token, translation 0.6–0.9), as hypothesised. The ordering across languages does not follow exposure: English recall is the dearest program, not the cheapest, in slots and in bits, and the result survives restricting the three recall test sets to the 4,212 items they share ($\hat{D}$ at $k = 4$: ru 0.049, de 0.079, en 0.122). The English LoRA floor is also the worst of the three (2.58 bits per token against 1.10 for Russian), so what is dear in English is not the language but the facts mLAMA asks about in English.

5.3 Neither nesting is cheaper, and sharing does not compress

Table 3 compares each group's R_{group} with the sum of its members' independent frontiers at $\hat{D} \leq 0.25$, the only level every cell reaches. Sharing does not compress. The task-centred hierarchy is at parity on translation and costs 1.9 times the independent programs on recall; the language-centred one is at parity on Russian and German and does not bring English recall under 0.25. Two effects make up the recall factor: at matched (k, β) a hierarchy costs 0.94–1.5 times the flat runs with the same hyper-parameters, so the shared centre buys nothing, and the frontier comparison adds the cost of one β serving every member, since the only task-centred run in which all three languages reach 0.25 has a centre (14,600 bits) that alone costs more than the three independent programs together. Nor is either nesting cheaper than the other: at matched (k, β) the language-centred code costs 0.88–1.24 times the task-centred one over all eight cells (Table 6).

The per-member costs are symmetric. Given the translation centre (604 bits), a target language costs 200–225 bits, less than half of a standalone translation program; given a language centre (830–900 bits), translation costs 201–257. Given the recall centre (14,600 bits), a language costs 3,200–3,950 bits, more than a standalone Russian or German recall program; given a language centre, recall costs 3,200–3,400, and for English no run reaches 0.25. Recall is the program that cannot be shared: it costs about 3,200 bits as a deviation from either kind of centre, and whichever group must carry it is the expensive one. In bits, then, the task given the language costs 0.9–1.0 times the language given the task on recall and 0.9–1.3 on translation, in agreement with the slot axis, where $\Delta k_{\text{lang}} = \Delta k_{\text{task}}$.

Under the preregistered protocol the task-centred nesting had appeared 37% cheaper than the language-centred one and 11% cheaper than independent programs (Table 5). Both margins came from language-centred recall deviations of 5,300–26,200 bits, which the corrected protocol brings down to 3,200–3,400; they were selection artefacts of the kind described in Section 6, not properties of the model.

group, $\hat{D} \leq 0.25$	hierarchy, bits (k, β)	independent, bits	ratio
translation (ru, de)	1,029 (4, 3)	978	1.05
recall (ru, de, en)	24,972 (4, 1)	12,917	1.93
continuation (ru, de, en; unmeasurable)	667 (4, 3)	3,724	0.18
Russian (translation, recall, continuation)	4,353 (2, 3)	4,204	1.04
German (translation, recall, continuation)	4,560 (2, 3)	4,227	1.08
English (recall, continuation)	not reached	9,188	–

Table 3: Task-centred (top) and language-centred (bottom) hierarchies against the sum of the members' independent frontiers, corrected protocol. The independent continuation values are from the preregistered runs, the only variational runs on continuation. Over all eight cells the task-centred code costs 26,700 bits against 17,600 for independent programs; the language-centred code never brings `recall-en` below 0.25.

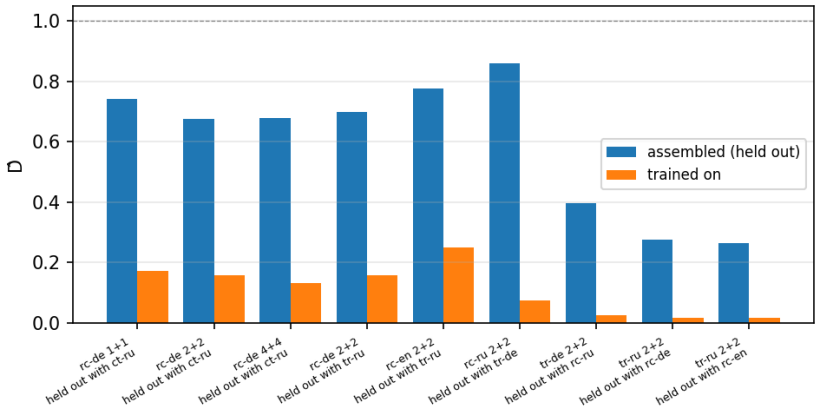


Figure 4: Assembled (held-out) against trained-on $\hat{D}$ of tied task and language parts, per hold-out.

5.4 Composition

Six tied parts trained on every cell match Design A at the same total slots on translation ($\hat{D}$ 0.017–0.027 against 0.005–0.007 at 2+2) and cost about twice its distortion on recall (0.08–0.25 against 0.05–0.12). Assembled zero-shot for a held-out cell (Figure 4), the parts close 60–73% of the gap between no prefix and the floor on translation (three hold-outs over two cells, $\hat{D}$ 0.27–0.40) and 14–32% on recall (six hold-outs over three cells, $\hat{D}$ 0.68–0.86). Task and language parts compose partially for translation and barely for recall.

5.5 Distance between programs

Warm-starting from a prefix of the same task costs 0.85 of training from scratch; from the same language or an unrelated cell, 0.95 (Figure 5, left). The silhouette of the task partition on the symmetrised distance is 0.004 and that of the language partition -0.09: the direction is the hypothesised one, the structure is absent, and with seven pairs per group at one seed the difference is not significant. The zero-shot block (right) is the strong signal: a prefix applied to any other cell is roughly as bad as, or worse than, no prefix ($\hat{D}$ 0.7–1.4, median 0.97, up to 2.5), and a recall prefix applied to continuation is catastrophic ($\hat{D}$ 5–25). The recall program overrides the model's default behaviour rather than adding to it.

5.6 Where the program is read

Patching block 0 alone recovers 70–87% of the effect for translation and 74–94% for continuation, but only 36–58% for recall, which has a secondary peak at block 17 (Figure 6). Single-block fractions summed over blocks come to 2.6 for translation and 3.0 for continuation, so their reading is redundant,

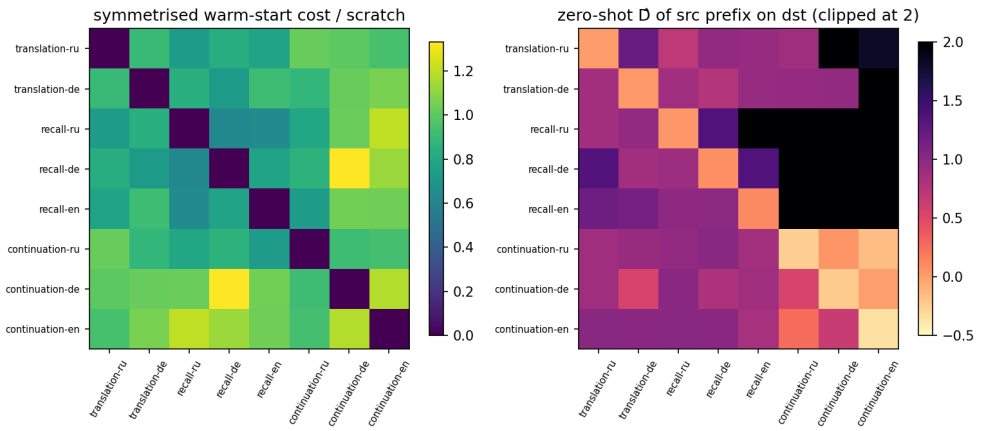


Figure 5: Left: symmetrised warm-start learning cost divided by the from-scratch cost. Right: $\hat{D}$ of a trained prefix applied unchanged to another cell, clipped at 2.

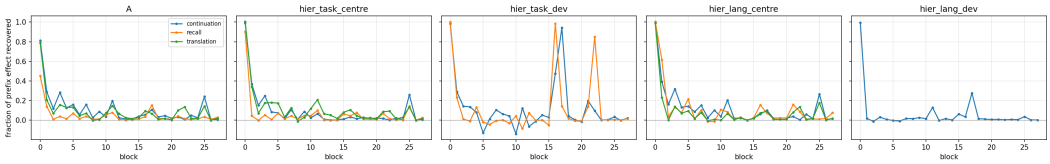


Figure 6: Fraction of a prefix's effect recovered by patching its positions into one block's attention input, mean over cells per task. Probes: `A` (Design A, $k = 4$); `hier_task_centre` and `hier_task_dev` (task centre; language deviation on top of it); `hier_lang_centre` and `hier_lang_dev` (language centre; task deviation on top of it). Fractions are undefined where the donor is not at least 0.1 bits per token better than the recipient.

but 1.3 for recall, which is close to additive. A translation program is copied into the token stream in the first block and not needed afterwards; a recall program is read in several places, deep in the network.

Inside a hierarchical prefix the parts differ by depth profile rather than by peak, which is block 0 for every probe. The task centre is read early (on recall, its mass over blocks 0–3 is 1.0 and over blocks 14–27 it is 0.5); the language deviation on top of it is read early and again deep (1.6 and 1.9, with block 17 alone recovering 0.94 of its effect), and a task deviation under a language centre behaves like the task centre. This is a split by part in the opposite direction to the language-early, task-middle expectation drawn from Dumas et al. [3]. The recall centre without its English deviation is worse than a random prefix on English, so the recall centre is a non-English program. These probes were run on the preregistered hierarchies; on the corrected ones the split replicates where the probe is defined, and a probe on posterior samples is implemented but not yet run.

6 The objective is right, the search is hard

The saved histories of the 27 task-grouped hierarchical runs under the preregistered protocol show three failures (Figure 7). With learning rate 0.05, two prior standard deviations per Adam step, the centre KL overshoots early, peaking at a median of 26,000 bits at $k = 2$ and 144,000 at $k = 8$ before the cosine schedule lets the KL term pull the means back. The checkpoint rule, which selects on the distortion of the posterior mean, then picks a step where the rate is above its final value, by a median factor of 1.1–1.2 and up to 20. And the posterior mean is not the coded object: on translation at $\beta = 3$ the mean scores $\hat{D}$ 0.35–0.56 in three of four runs while samples score 0.06–0.17, because the model was trained on samples.

The corrected protocol removes all three. The centre KL of the 12 corrected hierarchical runs peaks 7–12 times lower, then falls monotonically, and the selected checkpoint is within 2% of the final

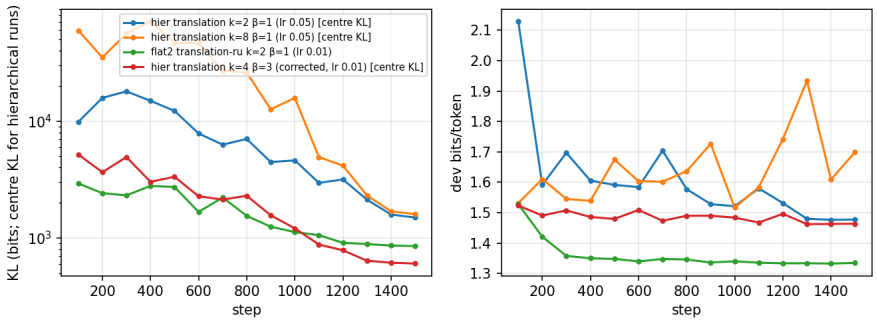


Figure 7: Centre KL (hierarchical runs) or total KL (flat2) and development loss against step for two preregistered-protocol hierarchical runs and two corrected-protocol runs (flat2 and hierarchical).

rate. The measured frontier drops on every cell, by 22% (`translation-ru`, `recall-ru`), 58–63% (`translation-de`), 20% (`recall-de`) and 71% (`recall-en`), with four runs per cell instead of nine, and the rates become monotone in β. The protocol also decides a result: the preregistered 37% advantage of the task-centred nesting (Section 5.3) does not survive it. Shaw et al. [12] report that on a parity task the same objective reaches 870 bits from a hand-compiled initialisation but 2.8×10^6 bits from random initialisation, and leave the optimiser as the open problem. At this scale the gap closes mostly by selecting on the objective and stepping at the scale of the prior.

7 Limitations

Seeds and budget. Two seeds were preregistered; the second seed of the corrected protocol reproduces the frontier within 4% on three cells and within 17–22% on the other two, with the same cell ordering (Table 7); the hierarchies and the distance matrix are one seed. Every corrected rate, flat or hierarchical, is still falling at the end of the 1500-step budget, so all are upper bounds, and the hierarchical comparisons rest on four runs per group and one τ.

Distortion is a teacher-forced code length. L scores the reference target token by token under the ground-truth prefix of the target. It is the quantity the description-length argument needs, but it is not generation quality: a program with low L may still decode poorly, and a program that produces a valid paraphrase is charged for every token that differs from the reference.

Bits depend on the prior. R is the code length under π_0, a diagonal Gaussian fitted to the embedding matrix. A different prior, such as the per-column mixture of Shaw et al. [12], changes every absolute number. The claims that rest on the bit unit are orderings and ratios between cells coded under the same prior, and the preregistered 37% margin shows how far a protocol change alone can move them.

Uncertainty over sampled prefixes. The reported distortion averages eight passes over the test set with a fresh sample per batch, which estimates $\mathbb{E}_{P\sim q}[L(P)]$ well but says nothing about the variance of $L(P)$ over single samples, and therefore nothing about the probability that one transmitted prefix misses a threshold. Several frontier points sit within two standard errors of their threshold on the test examples, so the exploratory 0.05 level in particular should be read as indicative.

Other. Continuation is unmeasurable because the frozen model already performs it. One corrected run (English centre, $k = 2$, $\beta = 1$) selected a transient dip in the development loss; selection on the objective is not immune to noise. The mechanism probes patch attention inputs at the prefix positions; a residual-stream patch would answer a slightly different question. The exposure proxy for the language ordering was not available, so that comparison rests on orderings over three languages.

8 Conclusion

Counting tokens misreads a prompt's complexity; counting bits under a bits-back code does not, and at that currency a frozen 1.5B model needs about 500 bits to be told to translate and 2,400 or more to be told to recall facts. Those bits are not organised as a cheap modifier on a shared core in either direction: a language given a task costs what a task given a language costs, the two nestings cost

the same, and a centre plus deviations costs no less than independent programs. The asymmetry is mechanistic: recall programs are language-specific, do not compose and are read deep, whereas translation programs are read in the first block. The measurement is only as good as the optimiser, and the largest single effect in this study is the one obtained by selecting checkpoints on the objective rather than on distortion. The research version runs the same code over the scale axis and twenty languages, charges $\log_2 |k\text{-grid}|$ bits for the choice of k, adopts the per-column mixture prior of Shaw et al. [12], and gives each member of a hierarchy its own β, with three seeds.

Reproducibility Code, result tables, per-example bits, training histories and prefix checkpoints are in the repository (branch `prefixlab`); `notebooks/01_figures.ipynb` regenerates every figure and table from the tables and `prereg.json`.

References

[1] Léonard Blier and Yann Ollivier. The Description Length of Deep Learning Models. In *Advances in Neural Information Processing Systems*, 2018.

[2] Adrian Cosma. Prompting Complexity: Shortest Prompts for Texts and Behaviors in LLMs. *arXiv preprint arXiv:2607.06145*, 2026.

[3] Clément Dumas, Chris Wendler, Veniamin Veselovsky, Giovanni Monea, and Robert West. Separating Tongue from Thought: Activation Patching Reveals Language-Agnostic Concept Representations in Transformers. In *Proceedings of ACL*, 2025.

[4] Roee Hendel, Mor Geva, and Amir Globerson. In-Context Learning Creates Task Vectors. In *Findings of EMNLP*, 2023.

[5] Geoffrey E. Hinton and Drew van Camp. Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights. In *Proceedings of COLT*, 1993.

[6] Nora Kassner, Philipp Dufter, and Hinrich Schütze. Multilingual LAMA: Investigating Knowledge in Multilingual Pretrained Language Models. In *Proceedings of EACL*, 2021.

[7] Daniel Khashabi, Xinxi Lyu, Sewon Min, Lianhui Qin, Kyle Richardson, Sean Welleck, Hannaneh Hajishirzi, Tushar Khot, Ashish Sabharwal, Sameer Singh, and Yejin Choi. Prompt Waywardness: The Curious Case of Discretized Interpretation of Continuous Prompts. In *Proceedings of NAACL*, 2022.

[8] Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of EMNLP*, 2021.

[9] Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of ACL*, 2021.

[10] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems*, 2023.

[11] Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.

[12] Peter Shaw, James Cohan, Jacob Eisenstein, and Kristina Toutanova. Bridging Kolmogorov Complexity and Deep Learning: Asymptotically Optimal Description Length Objectives for Transformers. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.22445.

[13] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2024.

[14] Elena Voita and Ivan Titov. Information-Theoretic Probing with Minimum Description Length. In *Proceedings of EMNLP*, 2020.

[15] Tu Vu, Brian Lester, Noah Constant, Rami Al-Rfou, and Daniel Cer. SPoT: Better Frozen Model Adaptation through Soft Prompt Transfer. In *Proceedings of ACL*, 2022.

[16] Ivan Vykopal, Simon Ostermann, and Marián Simko. Soft Language Prompts for Language Transfer. In *Proceedings of NAACL*, 2025.

[17] Chris Wendler, Veniamin Veselovsky, Giovanni Monea, and Robert West. Do Llamas Work in English? On the Latent Language of Multilingual Transformers. In *Proceedings of ACL*, 2024.

A Preregistered comparisons

H1	task ordering supported (recall far above translation); language ordering reversed on recall (English dearest), robust to common items; exposure proxy unavailable
H2	equal on the slot axis (1.8 slots each, 8 of 9 from one cell); in bits, equal under the corrected protocol (task given language 0.9–1.0× language given task on recall, 0.9–1.3× on translation); the preregistered 1.3–7.8× asymmetry on recall does not replicate
H3	translation composes partially (60–73% of the gap zero-shot), recall barely (14–32%); a shared centre saves no bits: under a common protocol the task-centred hierarchy costs 1.5× eight independent programs (parity on translation, 1.9× on recall) and the two nestings cost the same within 25%; Chinese not run
H4	silhouette 0.004 vs −0.09: task direction, no structure, not significant
H5	block 0 for both; the language deviation is read again at blocks 16–17 and the task centre is not: a split by part, opposite in direction to the hypothesis
H6	one vector suffices with the task prefix; per-token variant not computed; no rate term

Table 4: Outcome of each preregistered comparison.

B Hierarchy: detailed comparisons

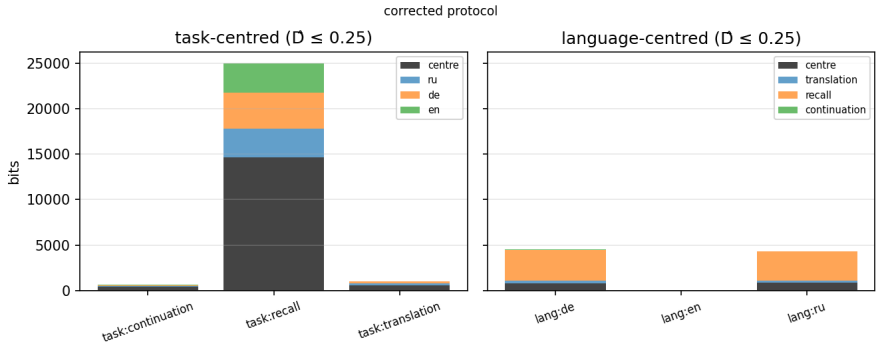


Figure 8: Code length at $\hat{D} \leq 0.25$ under the two nestings, corrected protocol: the centre is paid once per group, each deviation once per member; the cheapest run whose members all reach the level.

code for all eight cells, $\hat{D} \leq 0.25$	protocol	bits
centre per task + language deviations	prereg. selection	33,500
eight independent programs	prereg.	37,600
centre per language + task deviations	prereg. selection	53,500
centre per task + language deviations	corrected	26,700
centre per language + task deviations	corrected	not reached (ru, de: 8,900)
eight independent programs	corrected	17,600

Table 5: Three codes for the same eight cells under both protocols. The corrected independent row uses flat2 on the five measurable cells and the preregistered runs on continuation; under the corrected protocol the language-centred hierarchy does not get `recall-en` below 0.25 within the grid, so only its Russian and German groups are summed.

(k, β)	task-centred, bits (worst $\hat{D}$)	language-centred, bits (worst $\hat{D}$)	ratio
(2, 1)	24,591 (0.28)	30,448 (0.59)	1.24
(2, 3)	12,693 (0.56)	11,182 (0.48)	0.88
(4, 1)	28,659 (0.23)	30,532 (0.28)	1.07
(4, 3)	11,180 (0.28)	13,396 (0.37)	1.20

Table 6: The two nestings at matched hyper-parameters under the corrected protocol: three centres plus eight deviations either way, summed over all eight cells, with the worst member distortion. At matched (k, β) a hierarchy costs 0.94–1.5 times the sum of the flat2 runs with the same hyper-parameters; on recall the factor 1.93 of Table 3 decomposes as 1.12 (the matched excess at $(4, 1)$, 24,972 against 22,360) times 1.73 (one β for all members, 22,360 against 12,917). Under the preregistered rule a language given translation cost 485–662 bits and recall given a language 5,300–26,200.

C Input compressibility (exploratory)

With the $k = 4$ task prefix, one per-example vector replaces the input to within 10% of the real-input loss on every cell but `translation-ru`, which needs two; without the prefix, translation and continuation need two. There is no rate term on the vectors, so this is a slot count. On recall the bare model scores the target better with no input (3.1 bits per token) than with the real query (7.2), consistent with confident wrong answers once a subject is named.

D Run inventory and replication

273 Design A, baseline and floor rows (6 values of k, 2 seeds, 3 conditionings, 8 cells); 72 flat and 40 flat2 rows (two seeds); 144 hierarchical rows from 54 joint runs under the preregistered rule and 64 from 24 under the corrected protocol; 72 composition rows from 10 joint runs; 72 distance rows; 32 regime 2 rows; 1,120 mechanism rows, plus 1,120 mean-based mechanism rows kept as a caveated check; one 3000-step recall run. Training: 1500 steps, batch 16, AdamW, learning rate 0.05 (0.01 for flat2), 50 warm-up steps, cosine decay, gradient clip 1.0, development evaluation every 100 steps. LoRA floors: rank 16 on the attention projections, learning rate 2×10^{-4}, 1500 steps. Bootstrap intervals over test examples for the gain of the $\hat{D} \leq 0.10$ frontier runs over the no-prefix model, evaluated at the posterior mean (per-example bits are stored for the mean only): 0.52 ± 0.01 bits per token on `translation-ru`, 0.50 ± 0.03 on `translation-de` and 5.69 ± 0.07 on `recall-ru`.

cell	seed 0, bits at 0.25 (k, β)	seed 1, bits at 0.25 (k, β)	ratio
`translation-ru`	542 (4, 3)	534 (4, 3)	0.99
`translation-de`	436 (2, 3)	533 (4, 3)	1.22
`recall-ru`	2,419 (2, 3)	2,486 (2, 3)	1.03
`recall-de`	3,343 (4, 3)	2,783 (2, 3)	0.83
`recall-en`	7,155 (4, 1)	6,850 (4, 1)	0.96

Table 7: Replication of the corrected-protocol frontier with a second seed; each seed's frontier is an independent replicate of the whole run grid. The binding level is 0.25 for `recall-de` and `recall-en` and 0.10 for the other cells, where the two levels coincide.